

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 1 | AULA 01 -

INTRODUÇÃO AOS PRINCÍPIOS SOLID

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	10
REFERÊNCIAS	11

EMANDA

O QUE VEM POR AÍ?

Nesta aula, aprenderemos os princípios SOLID. Esses princípios são essenciais para qualquer pessoa desenvolvedora que deseja dominar os conceitos de design de software orientado a objetos com eficácia e proficiência.



HANDS ON

Nesta aula em vídeo, o professor apresentará os princípios SOLID através de exemplos que demonstram a aplicação desses conceitos, fundamentais na programação orientada a objetos.



SAIBA MAIS

Os princípios SOLID são um conjunto de diretrizes para a programação orientada a objetos que foram idealizadas para ajudar pessoas desenvolvedoras a criar sistemas mais robustos, flexíveis e fáceis de manter.

O acrônimo SOLID representa os cinco princípios fundamentais formalizados por Robert C. Martin, também conhecido como Uncle Bob. Estes princípios ajudam a abordar problemas comuns de design de software, como código rígido, frágil, não reutilizável e difícil de entender. Antes de avançarmos neste tema, vamos entender como o SOLID surgiu.

O conceito de SOLID surgiu da necessidade de resolver problemas recorrentes na engenharia de software. Robert C. Martin, um renomado consultor de software e autor, apresentou estes princípios no início dos anos 2000, como parte de seu trabalho para promover boas práticas de design de software.

Seu objetivo era criar uma base sólida para o desenvolvimento de software orientado a objetos que pudesse resistir à evolução e mudança dos requisitos ao longo do tempo. Isto é, parar com o fluxo de, a cada modificação no sistema, ser sugerida sua reescrita.

Mas quais são as vantagens de se utilizar o SOLID? Existem algumas vantagens e listaremos aqui as cinco principais:

1. **Manutenção facilitada:** ao seguir os princípios SOLID, o código torna-se mais modular, facilitando a manutenção e atualização. Mudanças podem ser implementadas em uma parte do sistema sem afetar outras partes, reduzindo o risco de introduzir bugs.
2. **Reutilização de código:** princípios como SRP e ISP promovem a criação de componentes específicos e independentes, que podem ser reutilizados em diferentes partes do sistema ou até mesmo em projetos diferentes.
3. **Testabilidade:** SOLID incentiva a criação de código desacoplado, tornando mais fácil escrever testes unitários eficazes. Isso garante que diferentes partes do sistema possam ser testadas isoladamente.

4. **Flexibilidade e escalabilidade:** OCP e DIP promovem a extensibilidade e a flexibilidade do sistema, permitindo que novas funcionalidades sejam adicionadas com o mínimo de alterações no código existente.
5. **Legibilidade e compreensão:** um código que segue os princípios SOLID é geralmente mais fácil de entender, pois cada classe e módulo possui responsabilidades claras e bem definidas.

Mas como não existe “bala de prata”, vejamos algumas das desvantagens de se utilizar o SOLID.

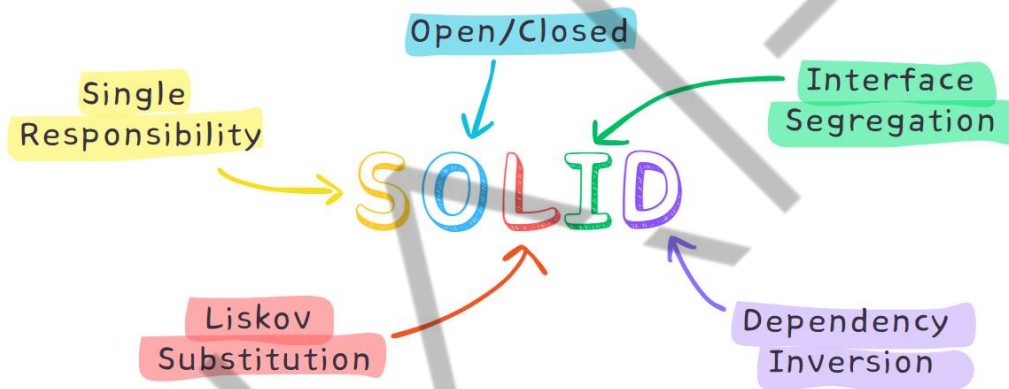


Figura 1 — Princípios SOLID
Fonte: elaborado pelo autor (2024)

Implementar o SOLID pode introduzir uma complexidade adicional no início do desenvolvimento, pois exige um planejamento cuidadoso e uma estrutura bem definida. Além disso, seguir princípios como SRP e ISP pode resultar em um número maior de classes e interfaces, o que pode ser visto como uma sobrecarga de código e estrutura.

Pessoas desenvolvedoras menos experientes podem também encontrar dificuldades para entender e aplicar os princípios SOLID de maneira eficaz, levando a possíveis mal-entendidos e implementação inadequada.

Por fim, em alguns casos a aplicação rigorosa dos princípios SOLID pode introduzir overhead, impactando o desempenho do sistema devido ao maior número de abstrações e camadas de indiretividade.

Agora que sabemos o que é o SOLID e vimos algumas das suas vantagens e desvantagens, vamos conhecer melhor cada uma das letras desse acrônimo:

- **Single Responsibility Principle (SRP):** cada classe deve ter apenas uma razão para mudar, simplificando assim a compreensão e a manutenção do código.
- **Open/Closed Principle (OCP):** entidades de software (classes, módulos, funções, etc.) devem estar abertas para extensão, mas fechadas para modificação, permitindo que o comportamento de uma entidade seja estendido sem alterar seu código-fonte.
- **Liskov Substitution Principle (LSP):** objetos de uma classe derivada devem ser capazes de substituir objetos de uma classe base sem comprometer a integridade da aplicação.
- **Interface Segregation Principle (ISP):** nenhum cliente deve ser forçado a depender de métodos que não utiliza, promovendo interfaces menores e mais específicas que são mais fáceis de implementar e manter.
- **Dependency Inversion Principle (DIP):** módulos de alto nível não devem depender de módulos de baixo nível. Ambos devem depender de abstrações.

Cases de uso do SOLID

1. Desenvolvimento de sistemas de e-commerce

Em um grande sistema de e-commerce, a aplicação dos princípios SOLID pode ser crucial para gerenciar a complexidade do sistema, especialmente quando se trata de funcionalidades como processamento de pedidos, pagamento e gestão de produtos.

Caso de uso:

Imagine que uma empresa de e-commerce aplicou o princípio da Responsabilidade Única (SRP) para separar as responsabilidades de diferentes partes do sistema. Dividiram a classe de processamento de pedidos em várias classes

menores, cada uma responsável por uma parte específica do processo, como cálculo de impostos, verificação de estoque e processamento de pagamento. Isso não apenas facilita a manutenção do sistema, mas também permite que diferentes equipes trabalhem em paralelo em diferentes partes do código.

2. Desenvolvimento de aplicações bancárias

Os sistemas bancários exigem um alto nível de segurança e robustez. Aplicar os princípios SOLID pode ajudar a garantir que o sistema seja extensível e seguro.

Caso de uso:

Vamos imaginar que um banco implementou o Princípio da Inversão de Dependência (DIP) para separar a lógica de negócios da lógica de acesso a dados. Neste caso, são introduzidas interfaces para operações de banco de dados, permitindo que a lógica de negócios dependa dessas interfaces em vez de classes concretas.

Isso facilita a substituição da camada de acesso a dados por uma nova tecnologia sem impactar a lógica de negócios, promovendo a flexibilidade e a evolução do sistema.

3. Desenvolvimento de jogos

Na indústria de jogos, a modularidade e a capacidade de extensão são cruciais para o desenvolvimento contínuo e atualização de jogos.

Caso de uso:

Imagine agora que uma empresa de desenvolvimento de jogos aplicou o Princípio Aberto/Fechado (OCP) para permitir a adição de novas funcionalidades aos seus jogos sem modificar o código existente. Ela cria uma arquitetura baseada em componentes, onde cada componente do jogo (como renderização, física, áudio) implementa uma interface comum.

Novos componentes podem ser adicionados ou estendidos sem alterar os componentes existentes, facilitando a expansão do jogo com novas funcionalidades e melhorando a experiência do usuário.

4. Desenvolvimento de aplicações web

Aplicações web modernas frequentemente necessitam de uma estrutura que permita fácil manutenção e evolução.

Caso de uso:

Este é um exemplo mais comum, por existirem muitas aplicações na WEB hoje. Imagine que uma empresa de software que desenvolve aplicações web aplicou o Princípio da Segregação de Interface (ISP) para criar interfaces mais específicas para diferentes tipos de usuários e funcionalidades. Em vez de uma única interface para todos os serviços, eles criaram interfaces menores e mais coesas, permitindo que cada serviço dependesse apenas dos métodos que realmente utilizava.

Isso resultou em um código mais limpo e fácil de entender, além de facilitar a adição de novas funcionalidades sem comprometer a integridade do sistema.

5. Desenvolvimento de APIs

APIs precisam ser flexíveis e adaptáveis para suportar diferentes clientes e evoluir com o tempo.

Caso de uso:

Uma empresa de tecnologia aplicou o Princípio da Substituição de Liskov (LSP) ao desenvolver uma API para um serviço de pagamento. Ela garantiu que todas as classes derivadas (como pagamentos via cartão de crédito, PayPal, etc.) pudessem substituir a classe base de pagamento sem alterar o comportamento esperado. Isso permitiu que novos métodos de pagamento fossem adicionados ao sistema sem a necessidade de modificar o código existente, garantindo a compatibilidade e a integridade do sistema.

Talvez você esteja com algumas dúvidas sobre como implementar o SOLID no seu dia a dia, correto? Então, não deixe de assistir a aula prática em vídeo, pois demonstraremos um passo a passo de como trabalhar com esses princípios que garantem a qualidade do nosso código.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você aprendeu a teoria dos princípios SOLID, que são a espinha dorsal do design de software orientado a objetos. Estes princípios oferecem um caminho para criar códigos limpos, organizados e resilientes, capazes de suportar mudanças e expansões com facilidade.

EMEND

REFERÊNCIAS

CASA do Desenvolvedor. **Princípios SOLID: Entendendo o que são e como podem revolucionar seu código.** 2020. Disponível em: https://blog.casadodesenvolvedor.com.br/principios-solid-o-que-sao/?gad_source=1&gclid=CjwKCAjwyo60BhBiEiwAHmVLJU450K2yVR8Zw95XfweT8uBUM4FhqJhALvwetmt2dGwGpS1cD0Q9VBoC2R0QAvD_BwE>. Acesso em: 06 ago. 2024.

MEDIUM. **O que é SOLID: O guia completo para você entender os 5 princípios da POO.** 2019. Disponível em: <<https://medium.com/desenvolvendo-com-paixao/o-que-%C3%A9-solid-o-guia-completo-para-voc%C3%AA-entender-os-5-princ%C3%ADpios-da-poo-2b937b3fc530>>. Acesso em: 06 ago. 2024.

MEDIUM. **Princípios SOLID.** 2024. Disponível em: <<https://medium.com/@milenafurutashishito/princ%C3%ADpios-solid-b2c5e770f654>>. Acesso em: 06 ago. 2024.

PALAVRAS-CHAVE

Palavras-chave: SOLID, SRP, OCP, LSP, ISP, DIP.

EXEMPLO



POSTECH